

ExamManager

AI-assisted exam marking for schools. Teachers photograph or upload students' answer scripts; an AI examiner marks each script against the school's own question paper and marking scheme; the system (never the AI) adds up the marks; and the teacher keeps the final say on every result.

Part 1: the problem and what the product can do

Part 2: how every feature works and how to use it

Contents

PART 1 · OVERVIEW

The problem

The solution

Capabilities

Design principles

Who uses it

Limits and requirements

PART 2 · USER GUIDE

Installing and starting

Teacher accounts

Signing in and navigating

Overview page

Setting up an exam

Adding scripts one by one

Uploading a whole class

The marking queue

How a script is marked

Results page

Changing a mark

Reports and printing

Spreadsheet export

Adding students later

Settings and clean-up

Administration

Configuration reference

Where data is stored

Security and privacy

Troubleshooting

Glossary

The problem and the product's capabilities

Why ExamManager exists, what it does, and what to expect from it before you install it.

1. The problem

Marking is one of the largest and least visible workloads in a school. Each term, every teacher marks several rounds of tests, mid-terms, end-of-term exams and mocks, for several classes and streams, often with 40-80 students per stream. A single teacher can face hundreds of handwritten scripts in one week, each of which has to be read against a marking scheme, scored question by question, totalled, converted to a percentage, entered into a class list, and commented on.

What goes wrong today

- **Time.** Marking a full class set by hand takes days. That time comes out of lesson preparation, remedial teaching and rest, and results reach students late, when the feedback is least useful.
- **Consistency.** The same answer can get different marks depending on whether it was the fifth or the fiftieth script, which teacher marked it, or how tired they were. Standards drift across a pile and between streams.
- **Arithmetic and transcription errors.** Adding up stage marks, converting raw marks to percentages, and copying totals into a mark sheet are all manual steps where mistakes creep in and are hard to spot.
- **Thin feedback.** Under time pressure, written comments shrink to a tick, a number, or "good work". Students and parents rarely get a clear picture of strengths and gaps.
- **Poor class-level insight.** Mean, highest, lowest and score distribution for a class are rarely calculated, so weak topics across a class go unnoticed.
- **Paper-based records.** Scripts are handed back or lost, and there's no searchable history of which student got what mark, when, and why.

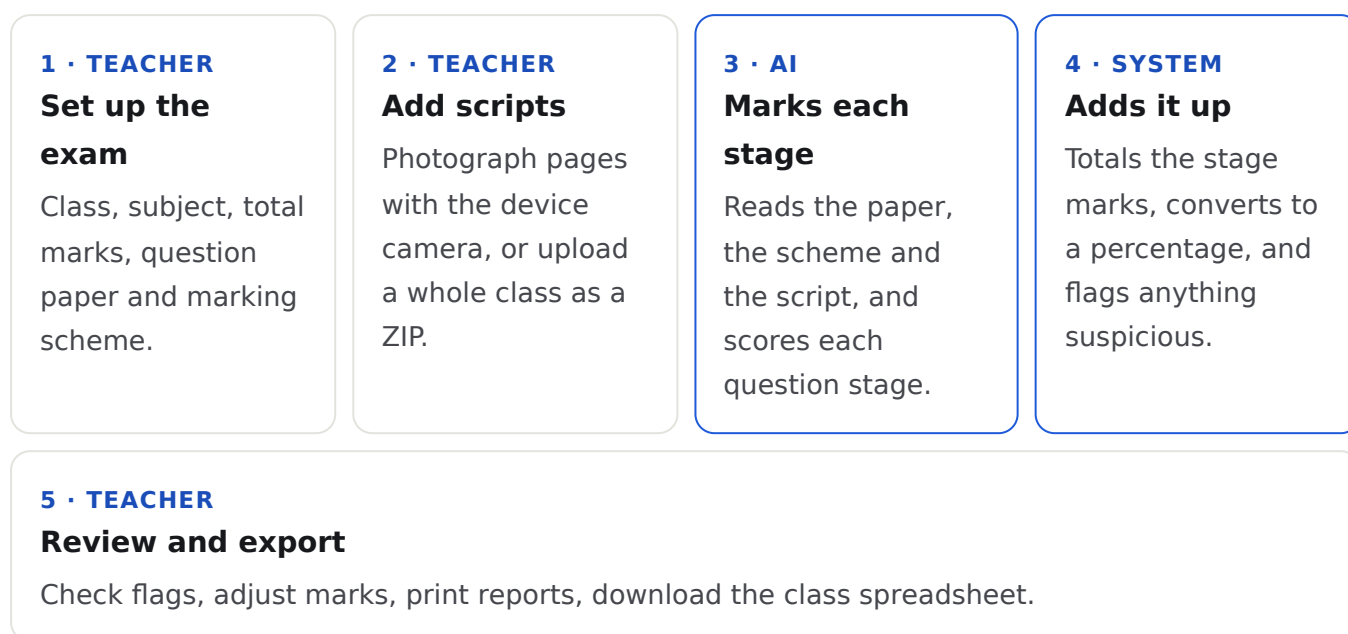
Why a generic AI chatbot doesn't solve it

Pasting scripts into a chatbot one at a time is slow, loses the marking scheme between conversations, has no record-keeping, does arithmetic inconsistently, can't process a whole class, and gives the teacher no structured way to check or correct results.

Schools need a tool built around the actual marking workflow: *set up the paper once, feed in a whole class, review, correct and export.*

2. The solution in one picture

ExamManager is a web application that runs on one computer at the school. Teachers use it from any phone, tablet or laptop on the same network, in a normal web browser. Nothing needs installing on the teachers' devices.



Marking happens in the background. A teacher can keep photographing the next student, or close the browser entirely, while earlier scripts are being marked.

3. Capabilities

Exam setup

One setup per paper

Enter year, term, class, stream, subject, exam name and total marks once. Every student added to that session is marked against the same paper and scheme.

Paper and scheme as photos or PDFs

Upload or photograph the question paper and marking scheme (PDF, JPG, PNG, WEBP), plus optional typed marking notes for the AI.

Total-marks check

If the marking scheme has readable text, the system checks whether its stage marks add up to the total you entered and warns you if they don't.

Built-in curriculum lists

Ready-made lists of classes (S1–S6, P1–P8), terms and 38 subjects, including local languages, so setup is a few taps.

Capturing scripts

Camera capture

Photograph pages straight from a phone or tablet camera inside the browser, choosing between front and back cameras.

File upload

Choose photos or a PDF from the device, or drag and drop them. PDFs are split into page images automatically.

Blur and lighting check

Every photo is checked for blur, darkness and over-exposure. Individual uploads are blocked until bad pages are retaken, so the AI never marks an unreadable page.

Whole-class ZIP or folder upload

Upload one ZIP (up to 500 MB), or pick a folder, containing one sub-folder per student. The system reads student IDs and names from folder names and shows a review screen before anything is marked.

AI marking

Stage-by-stage scoring

The AI reads the whole marking scheme, lists every scoring stage, and marks only the stages the student attempted, never above each stage's maximum.

System arithmetic

The AI returns only a list of stage marks. ExamManager adds them up and computes the percentage, so there are no AI arithmetic errors.

Examiner's report

A 220–280 word written report per student covering overall performance, strongest and weakest areas with examples, and practical recommendations.

Automatic sanity checks

Results where the stage marks exceed the paper total, include negative numbers, can't be read, or where the report was cut off are flagged **Needs review** instead of being quietly accepted.

Speed and reliability

Background queue

Scripts wait in a queue and are marked in parallel. The queue can be paused and resumed, and it survives restarts: unfinished jobs are picked up again when the app starts.

Multiple AI keys

Up to five (or more) Gemini API keys can be configured. Each adds a marking worker. If one key hits its quota, work moves to another key automatically.

Automatic retries

When the AI service is busy or the network drops, jobs retry with growing, randomised waits. Invalid keys are set aside rather than failing students.

Clear failures

A script that can't be marked is shown as **Failed** with a plain-language reason, and a whole session's failures can be retried in one click.

Review, reporting and export

Results browser

All sessions in one place, with search, filters by term, class and subject, and tabs for Needs attention, In progress and Complete.

Teacher override

Correct a student's name, ID or final mark. The AI's original mark is always kept on record, and a teacher's mark is never overwritten by the AI.

Student and class reports

Printable A4 student reports (marks by stage, percentage, comments) and a class report with mean, mode, lowest, highest and a score distribution chart.

Spreadsheet export

Download the class list as Excel (.xlsx) or OpenDocument (.ods), choosing which columns to include, including AI mark vs teacher mark and marks per stage.

Administration

Teacher accounts

Every page requires sign-in. Accounts are created from the command line with hashed passwords, and sign-in attempts are rate-limited.

Backups

One command zips all scripts, results, accounts and settings, keeping the newest 7 backups. It can be scheduled daily.

Automatic clean-up

Optional: delete answer-script photos 48 hours after upload to save disk space and limit how long student work is kept. Marks and feedback are kept.

Benchmark tool

Re-marks a fixed set of students several times to measure consistency, and gives a pass/fail accuracy gate before any AI setting is changed.

4. Design principles

- **The teacher has the final say.** The AI proposes; the teacher reviews and can override any result. Overrides are permanent and the AI's original mark is kept alongside for accountability.

- **The system, not the AI, does the maths.** The AI never adds up totals or computes percentages. It only reports marks per stage.
- **Marked against your scheme.** The AI marks against the school's own question paper and marking scheme, which are sent with every script. No result is cached or reused; every marking is fresh.
- **Flag, don't hide.** Anything that looks wrong is marked *Needs review*, with the reason stated. Nothing is silently capped or corrected.
- **Nothing is lost silently.** Re-uploaded pages are moved aside, not deleted. Overwritten sessions are kept as backup folders. Previous marks are kept in a history when a student is re-marked.
- **Works on school infrastructure.** One computer, the school Wi-Fi, ordinary phones. No database server to maintain; data is stored as plain files.

5. Who uses it

Role	What they do in ExamManager
Teacher	Sets up exams, captures or uploads scripts, reviews and corrects results, prints reports, downloads class spreadsheets.
Head of department / Director of studies	Browses results across classes and subjects, reads class statistics, checks flagged results before they're shared.
Administrator (IT or designated staff)	Installs and starts the app, adds AI keys, creates teacher accounts, runs backups, enables clean-up, runs the benchmark before changing AI settings.

In the app itself every signed-in user has the same teacher permissions. Administrative tasks are done on the host computer, not from the browser.

6. Limits and requirements

Item	Requirement or limit
Host computer	Windows, macOS or Linux with Python 3.11 or newer (tested on 3.12 and 3.14). Needs internet access to reach the Google Gemini API.
Teachers' devices	Any modern browser on the same network. Camera capture needs a device with a camera and permission to use it.
AI service	At least one Google Gemini API key. Each key should come from a different Google Cloud project , because quota is counted per project.
File types	JPG, JPEG, PNG, WEBP and PDF.
Batch upload size	ZIP up to 500 MB uncompressed. Batches over 1,000 pages ask for extra confirmation.
Total marks	Greater than 0, up to 10,000. Default is 100.
Student IDs	Letters, numbers, spaces, dots, dashes and underscores, up to 64 characters.
Marking time	Usually under a minute per script. Several scripts are marked at once (one per key by default).

AI marking is an assistant, not a replacement.

Handwriting that is hard to read, diagrams, and open-ended answers can be marked imperfectly. Teachers should review flagged results and spot-check others before results are shared with students or parents.

Product guide: how every feature works and how to use it

Sections 7–9 are for the administrator setting the system up. Sections 10–21 are for teachers using it day to day. Sections 22–27 are reference material for administrators.

7. Installing and starting ExamManager (administrator)

1 **Install Python 3.11+** on the computer that will host the app, then open a terminal in the project folder.

2 **Create a virtual environment and install dependencies:**

```
python -m venv .venv
# Windows:      .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
```

3 **Create the `.env` file.** Copy `.env.example` to `.env`, then:

- Set `SECRET_KEY` to a long random value. Generate one with `python -c "import secrets; print(secrets.token_hex(32))"`.
- Paste your Gemini API keys into `GEMINI_API_KEY_1` ... `GEMINI_API_KEY_5`. Blank slots are ignored.

4 **Create at least one teacher account** (see section 8).

5 **Start the app:**

```
python app.py
```

The console prints the addresses to use, for example:

 ExamManager running on `http://localhost:5000`
On other devices on this network: `http://192.168.1.20:5000`

- 6 Open it from another device.** On a phone or laptop connected to the same Wi-Fi, open the `http://192.168.x.x:5000` address. If it doesn't load, allow Python through the host's firewall on port 5000 (on Windows, choose *Private networks* when asked).

What happens at start-up

- Warnings are printed if `SECRET_KEY` is not set, if there are no teacher accounts, or if no Gemini keys are configured (marking is disabled without keys).
- Unconfirmed batch uploads older than 24 hours are removed.
- If automatic clean-up is on, old script photos are deleted (see section 21).
- Marking jobs that were waiting or running when the app last stopped are put back in the queue.
- The server starts (waitress, one process, 8 threads by default).

Keep the host running

Marking happens on the host computer. If it sleeps or shuts down, marking pauses and resumes when the app is started again. Set the host's power settings so it doesn't sleep during marking.

8. Managing teacher accounts (administrator)

Accounts are managed from a terminal on the host, in the project folder. Passwords are typed at a hidden prompt (at least 8 characters) and stored only as salted hashes in `users.json`. Changes take effect immediately; no restart is needed.

Task	Command
Add a teacher (or reset their password)	<code>python manage_users.py add nakato --name "Mrs Nakato"</code>
Change a password	<code>python manage_users.py passwd nakato</code>
Remove a teacher	<code>python manage_users.py remove nakato</code>
List all teachers	<code>python manage_users.py list</code>

The display name is shown in the sidebar and used in the Overview greeting. Teachers who forget their password should ask the administrator to run `passwd` for them.

9. Signing in and finding your way around

Open the address the administrator gave you and sign in with your username and password. Sign-ins last 12 hours. After 8 wrong attempts from the same device within 5 minutes, sign-in is blocked for a few minutes.

If your sign-in expires while you are working, the app tells you to refresh the page and sign in again. You return to where you were.

The sidebar

Item	What it's for
Overview	Home dashboard: totals, recent sessions, the marking queue and results needing attention.
Mark an exam	Starts a new marking session (exam setup).
Results	Every session and student result, with reports and exports.
Current session card	Appears while you're working on an exam. Shows its subject, class and term, with an <i>Add scripts</i> link to go back to uploading.
Settings	Appearance, automatic clean-up, and read-only system information.
Help	A short in-app guide, the meaning of result labels, and common questions.
Dark mode / Sign out	Switch theme (remembered on this device) and sign out.

On a phone, the sidebar is hidden behind the menu button (☰) in the top-left corner.

10. The Overview page

The Overview is the first page after signing in. It updates itself every 5 seconds while marking is running, and every 20 seconds when idle.

- **Summary tiles:** *Exam sessions* (and how many are set up but still awaiting scripts), *Scripts marked* out of uploaded, *Average score* across marked scripts, and *Needs attention* (results to review plus failed scripts).
- **Recent sessions:** the latest 8 exams with progress ("12 of 40 marked"), average and status. Click a row to open it in Results.
- **Marking queue:** how many scripts are being marked, waiting and failed; the students currently being marked (with retry counts); how many scripts can be marked at once; and how many AI keys are resting after hitting a limit. If no keys are configured it says so.
- **Needs attention:** sessions with results to check before sharing.

- **New marking session** button to start setting up an exam.

On a brand-new installation the Overview shows a three-step getting-started panel instead.

11. Setting up an exam (Step 1 of 2)

Choose **Mark an exam** in the sidebar or **New marking session** on the Overview. A summary panel beside the form shows what you've filled in and a checklist of what's still missing.

Exam details (all required)

Field	Notes
Year	For example 2026.
Term	Term 1, Term 2 or Term 3.
Class	S1-S6 or P1-P8.
Stream	Free text, for example A, B or East.
Subject	Chosen from the built-in list of 38 subjects.
Exam name	Free text, for example Mid-term, End of term, Mock.
Total marks	The most marks a student can get on this paper. Used by the system to turn raw marks into a percentage.

Stream, subject and exam name can't contain `/` or `\`.

Question paper and marking scheme

For each, choose **Upload files** (choose or drag in PDFs or photos) or **Camera** (photograph each page in the browser; pick a camera if the device has more than one). Add pages **in order**. The file list shows each page with its number and whether it came from the camera or a file; you can remove any page.

- **Question paper:** every page of the paper the students sat.

- **Marking scheme:** the official marking guide with marks for each question. Use the clearest copy available; the AI scores each question against it, so a legible scheme gives more consistent marks.
- **Extra marking notes (optional):** typed guidance given to the AI with the scheme for every student, for example *"accept equivalent units; ignore spelling in Section B"*.

Press **Save and continue**.

After saving: the total-marks check

If the marking scheme is a text PDF or you typed notes containing mark allocations like (5 marks) , [3 marks] or Total: 50 marks , the system adds them up. If they clearly disagree with the total you entered, a **Check the total marks** dialog appears with the suggested total. Choose **Go back and edit** to correct it, or **Keep and continue**. Photographed schemes can't be read this way and skip the check.

If the exam already exists

An exam is identified by year + term + class + stream + subject + exam name. If one with the same details already has students, a **This exam is already set up** dialog appears:

- **Keep existing** cancels. To add students to the existing exam, go to Results and use Add students instead.
- **Replace it** starts a new, empty session. The old one is not deleted; it's kept on the server as a backup folder and no longer appears in Results.

Re-running setup on an exam replaces its question paper and/or scheme only when new ones are provided, so stale pages from a longer old paper are never left behind.

Next you are asked **How will you add the scripts?**: one student at a time, or a whole class from a ZIP. You can switch between them at any time in the session.

12. Adding scripts one student at a time

Best for phones and tablets, and for capturing scripts as they're handed in.

- 1 **Enter the Student ID and Full name.** Both are required.
- 2 **Add the answer pages in order**, using either:

- **Camera:** point at the page and tap capture. Each photo is added as the next page. Pick a different camera if needed.
- **Upload files:** choose or drag in photos or a PDF. Select several at once, in page order.

3 Check the page gallery. Each page is checked for sharpness and brightness as soon as it's added. Pages that need retaking are highlighted, and the counter says how many. Remove and retake them.

4 Press Upload and mark. The pages are checked again on the server. If any page is too blurry, too dark or washed out, nothing is saved and you're told which pages to retake. Otherwise the student is saved and joins the marking queue, and the form clears for the next student.

Tips for good photos

- Lay the page flat, in good even light, with the whole page in view.
- Avoid shadows from your hand or phone, and glare from overhead lights.
- Hold steady; tap to focus before capturing.

Re-uploading and ID rules

- Uploading again with the **same ID and same name** replaces that student's pages and re-marks them. The old pages are moved aside to a `previous_uploads` folder, never deleted.
- If the ID already belongs to a **different name** in this exam, the upload is refused, to avoid overwriting another student's work. Check the ID.
- If a teacher has already set a final mark for this student, the new pages are saved but the AI does not re-mark them.
- If the student is still waiting in the queue, the waiting job is updated rather than duplicated.

The queue panel on this page

Beside the form, a panel lists the students you've uploaded in this exam with their live status and overall progress. From here you can **Pause marking** / resume, go to **See results**, or **Finish this exam** (which closes the current session in your browser; marking still continues). If the connection drops, the panel says so; marking continues on the server and the list catches up when you're back online.

13. Uploading a whole class (ZIP or folder)

Best for scripts that have already been scanned, for example on a school photocopier.

Arrange the files

Put each student's pages in their own folder named with the **student ID, then their name**. The ID and name can be separated by an underscore, dash or space.

```
S2B_Physics/  
├─ 001_John Doe/  
│   ├─ page1.jpg  
│   └─ page2.jpg  
├─ 002_Mary Akello/  
│   └─ script.pdf  
└─ 003_Peter Ouma/  
    └─ p1.jpg
```

- Pages can be JPG, PNG, WEBP or PDF and are read in **number order** (page2 comes before page10).
- One wrapping folder around everything (like `S2B_Physics/` above) is fine; it's stripped automatically.
- A folder with only an ID (e.g. `005`) is accepted, but the student is flagged because the name is missing.
- Files outside a student folder, nested ZIP/RAR files, unsupported file types and system files (`.DS_Store` , `Thumbs.db` , `__MACOSX`) are skipped with a note.

Upload

- 1 Choose **ZIP file** and pick or drag in the ZIP (up to 500 MB), *or* choose **Folder** and pick the class folder; the browser packages it into a ZIP for you.
- 2 Press **Check files**. The system reads each student's folder and checks photo quality. Large classes can take a minute.

Review before marking

Nothing is marked until you confirm. The review screen shows totals (students, pages, ready with no issues), an overall banner, and a table of every student:

Status	Meaning	What happens on Start marking
Ready	Read cleanly.	Added and marked.
Check	Added, but something needs a look: name missing, a page looks blurry, or the same ID appeared in two folders (pages combined).	Added and marked. Blurry pages may be under-marked.
Skipped	No valid, readable pages were found.	Not added.

Open **Show details** to see every warning. Batches over 1,000 pages need an extra tick to confirm. Then press **Start marking**, or **Choose other files** to start over (the uploaded batch is discarded).

The whole batch is blocked if the ZIP is empty, password-protected, would expand beyond 500 MB, looks like a "zip bomb", or contains no identifiable student folders. The message says what to fix.

On confirmation, students whose ID already belongs to a different name in this exam are not added, and students with a teacher's final mark keep it. The result message lists both.

Unlike individual upload, batch upload doesn't block blurry pages.

They're flagged so you can decide. If a student's result looks low, check their pages and re-upload that student individually.

14. The marking queue

Every uploaded student becomes a *job* in a single queue shared by everyone using the app. Workers take jobs in order and mark them in parallel: one worker per AI key by default (more if the administrator sets `MARKING_WORKERS_PER_KEY`).

Job state	Meaning
Waiting	In the queue, not started yet.
Marking	A worker is marking it now.
Retrying	The AI service was busy or unreachable; it's waiting and will try again automatically (10 s, 20 s, 40 s ... up to 2 minutes apart).
Completed	Result saved.
Failed	Couldn't be marked; a reason is saved on the student.
Cancelled	Removed from the queue before marking started.

- **Pause / resume:** pausing stops new jobs from starting; jobs already being marked finish.
- **Close the browser freely:** marking runs on the host, not in your browser.
- **Restarts are safe:** waiting and running jobs are saved to disk and resumed when the app starts again.
- **AI quota:** if a key hits its limit it rests (for as long as Google asks, or an hour for a daily limit) and work moves to another key. If every key is out for more than 15 minutes, jobs fail with *"Daily AI limit reached"*. Use **Retry failed** later.

15. How a script is marked

Understanding this helps teachers judge when to trust a result and when to check it.

- 1 **Inputs are assembled.** The question paper, marking scheme, any extra notes and the student's pages are sent to the AI, each clearly labelled (for example "QUESTION PAPER — part 1 of 2", "STUDENT ANSWER SCRIPT"). Large submissions are uploaded through Google's file service automatically. Nothing is compressed in a way that could make handwriting harder to read.
- 2 **The AI reads the paper and the whole scheme first.** It notes the exam instructions (compulsory questions, "answer any three from Section B", etc.) and builds an ordered list of every scoring stage, such as Q1(a), Q1(b), Section B Q3, with each stage's maximum.

- 3 **It marks every attempted stage in order.** Stages the student didn't attempt are left out entirely (not given zero). Each attempted stage gets marks up to its own maximum, with partial credit where the scheme allows, and equivalent wording accepted.
- 4 **It returns a plain list of numbers** (SCORES: 4, 2, 5, 3) and a 220–280 word examiner's report. It never states a total or percentage.
- 5 **ExamManager does the arithmetic.** $\text{Percentage} = \text{sum of stage marks} \div \text{total marks for the paper} \times 100$.
- 6 **ExamManager checks the result** and marks it **Needs review** if: some stage scores couldn't be read; a stage score is negative; a single stage is larger than the whole paper; the stages add up to more than the paper total; or the report was cut off. The score is saved as-is (never silently capped) with the reason, so a teacher decides.
- 7 **Everything is recorded.** Every AI reply is saved per student in an `ai_runs` folder with the model used and timing. If a student is re-marked, the previous mark is kept in a history.

If every page is blank or unreadable, the AI says so and the student is marked **Failed** with "Pages unreadable or blank — re-capture and upload again".

16. The Results page

Results lists every exam session, newest first. Use the ↻ button in the top bar to reload.

Finding a session

- **Search** by subject, class or student name.
- **Filter** by term, class and subject; *Clear filters* resets them.
- **Tabs:** All · Needs attention (sessions with results to review or failures) · In progress (not everyone marked yet) · Complete. Each tab shows a count.
- Long lists show more with *Show more sessions*.

Inside a session

Click a session to expand its student list. Columns can be sorted by clicking their headings. Each student row shows ID, name, pages, score and status, with **Report** and

Edit buttons. Edited marks show a  pencil; hover it to see the AI's original mark.

Result labels

Label	Meaning	What to do
Marked	Marked by the AI with no problems found.	Spot-check as usual.
Needs review	Marked, but something looked wrong. The reason is shown.	Check the script and set the correct mark with Edit before sharing.
Failed	Couldn't be marked. The reason is shown next to the student.	Fix the cause, then use Retry failed.
Waiting	Uploaded and waiting in the queue.	Nothing; it will be marked.
Edited	A teacher changed the mark. The AI's mark is kept on record.	Nothing.

Scores are coloured by band: **70% and above**, **50-69%**, **below 50%**.

Session actions

- **Retry failed** (shown when a session has failures): re-queues every failed student in that session who still has pages and hasn't been given a teacher's mark.
- **⋮ menu**: Download spreadsheet · Class report and statistics · Print all student reports · Add students.

17. Changing a mark, name or ID (teacher override)

- 1 In Results, open the session and press **Edit** on the student's row.
- 2 Change the name, ID and/or mark. The mark is entered as a **percentage (0-100)**.
- 3 Press **Save** and confirm in the *Change this mark?* dialog.

- Your mark becomes the final mark everywhere: student report, class report and spreadsheet.
- The AI's original mark, name and ID are kept permanently, even after several edits. The student report notes "Mark set by a teacher. The AI's original mark was ...%".
- A *Needs review* result becomes *Marked* once a teacher sets the mark.
- A student with a teacher's mark is **never re-marked by the AI**, even if pages are re-uploaded or a marking job finishes afterwards.
- Changing the ID doesn't move the student's files; re-marking still finds their pages. An ID already used by another student in the session is refused.
- Who made the change and when is recorded.

18. Reports and printing

Student report

Press **Report** on a student row. The examiner's report shows the student's name, ID, subject, exam and term; **marks obtained / total** and the **percentage; marks by stage**; the **examiner's comments**; strengths and areas to improve where recorded; and a note if a teacher set the mark. Failed or flagged students carry a clear banner saying the report is not a final result. Press **Print** for an A4 layout (use "Save as PDF" in the print dialog to get a PDF).

Class report and statistics

From the session's ... menu choose **Class report and statistics**. It contains:

- A **summary**: number marked, average, and how many scored 70%+, 50-69% and below 50%, with a list of results flagged Needs review.
- **Statistics**: mean, mode, lowest, highest, students and marked. Unmarked students are left out and the report says how many.
- **Check before sharing**: each flagged student with the reason.
- A **score distribution** chart in five bands (0-19%, 20-39%, 40-59%, 60-79%, 80-100%).

These figures are plain arithmetic over the saved scores. No AI is involved. Press **Print** to print it.

Print all student reports

From the ... menu choose **Print all student reports** to print every student's report in the session in one go, one per page.

19. Downloading the class spreadsheet

From the session's ... menu choose **Download spreadsheet**. Tick the columns you want, choose the file type, and press **Download**.

Column	Default	Contents
Student ID	On	The student's ID.
Student name	On	The student's name.
Total score (%)	On	The final mark (teacher's mark if edited).
Feedback	On	The examiner's report text.
Status	On	Marked, Needs review, Failed, etc.
AI score (%)	Off	The AI's original mark.
Teacher's mark (%)	Off	The teacher's override, if any.
Difference	Off	Teacher's mark minus AI mark.
Edited by teacher	Off	Whether the mark was overridden.
Marks per stage	Off	One column per marking stage.

File types: **Excel (.xlsx)** or **OpenDocument (.ods)** for LibreOffice. The page count is always included. Students are sorted by ID the way a person reads them (S2 before S10).

20. Adding students to an exam later

If some scripts arrive after the rest, or you finished a session early:

- 1 Go to **Results** and find the session.

- 2 Open its ... menu and choose **Add students**.
- 3 Pick **One student at a time** or **Whole class from a ZIP file**.

The session becomes your current session again. You don't need to re-upload the question paper or marking scheme; they're read from the session's folder for every new student.

21. Settings and automatic clean-up

Appearance

Light or dark theme. Saved on this device only.

Automatic clean-up

When turned on, answer-script photos (student page folders) older than **48 hours** after upload are **permanently deleted each time ExamManager starts**. Marks, feedback and exported spreadsheets are kept; only the page photos go, so those students **can't be re-marked** afterwards.

- Turning it on requires confirming a warning dialog (*Turn on clean-up / Keep it off*).
- **Scripts that would be deleted now** previews how many folders and MB are past the limit.
- **Deletion history** lists the last 50 deletions: student, session, time and size.

Before turning clean-up on

Make sure scripts are reviewed and any re-marking is done within 48 hours of upload, and that backups are running if you need to keep the photos.

System (read-only)

Shows the AI model in use (and fallback model, if set), how many AI keys are configured and how many are resting, and how many scripts can be marked at once. These are changed by the administrator in `.env`.

22. Administration tasks

Backups

```
python tools/backup.py          # keeps the newest 7
python tools/backup.py --keep 14 # keep 14 instead
```

Creates `backups/exammanager-backup-YYYYMMDD-HHMMSS.zip` containing `uploads/` (all sessions, scripts and results), `users.json`, `settings.json`, `deletion_log.json` and `pending_jobs.json`. Photos are stored byte-for-byte. Unconfirmed batch uploads are skipped.

Schedule it daily. On Windows, create a Task Scheduler task that runs `.venv\Scripts\python.exe tools\backup.py` with the project folder as *Start in*. On macOS/Linux, add a cron entry. Copy backups off the host computer regularly.

To restore, stop the app, unzip a backup into the project folder (replacing `uploads/` and the JSON files), and start the app again.

Adding AI capacity

- Add keys to `GEMINI_API_KEY_1 ... _5` in `.env` and restart. Each key from a **separate Google Cloud project** adds one marking worker. Several keys from one project share one limit and add no speed. (More slots, or a comma-separated `GEMINI_API_KEYS`, are also accepted.)
- Paid-tier keys with high rate limits: set `MARKING_WORKERS_PER_KEY=2` (or more) to mark several scripts per key at once.
- An invalid or expired key is set aside for 6 hours and logged by slot number; replace it in `.env`.

Running the benchmark (before changing AI settings)

The benchmark re-marks a fixed set of students several times with the real marker and measures consistency. It never changes any session.

```
copy tools\benchmark_set.example.csv benchmark_set.csv # macOS/Linux: cp
# edit it: one row per student – session folder, student ID, optional teacher
mark
python tools/benchmark.py --set benchmark_set.csv --label baseline
# after changing a setting:
```

```
python tools/benchmark.py --set benchmark_set.csv --label after --compare
benchmarks/<baseline folder>
```

Results go to `benchmarks/<time>_<label>/` (`results.csv` , `summary.txt` , `summary.json` , raw AI records). `--compare` prints **ACCURACY GATE: PASS/FAIL**: the set's average score must not move by more than 2 percentage points, and the round-to-round spread (standard deviation) must not get worse. Only adopt a new model or setting if the gate passes.

Monitoring

- `logs/marking.log` : one line per marking job with timings, tokens used, retries and outcome.
- `.../student_<id>/ai_runs/*.json` : every AI reply for a student.
- The Overview queue panel and Settings → System show live worker and key status.

23. Configuration reference (`.env`)

Restart the app after changing `.env` .

Setting	Default	Purpose
SECRET_KEY	(must set)	Secures sign-in sessions. Long random value.
GEMINI_API_KEY_1 ... _5	blank	AI keys. One marking worker per key.
GEMINI_MODEL	gemini-3.6-flash	Model used for marking. Change only after benchmarking.
GEMINI_FALLBACK_MODEL	blank	Used only when the main model is overloaded. Blank = never switch.
GEMINI_THINKING_LEVEL	blank	minimal / low / medium / high. Lower is faster and cheaper. Benchmark first.
GEMINI_MAX_OUTPUT_TOKENS	32768	Output budget per reply.
GEMINI_RETRY_OUTPUT_TOKENS	65536	Larger budget used once when a reply comes back cut off.
PAGE_EXIF_ROTATE	0	1 = rotate phone photos upright before sending to the AI (originals untouched). Benchmark first.
PAGE_MAX_EDGE	0	Downscale pages larger than this many pixels on the long edge (e.g. 2400). 0 = off. Benchmark first.
MARKING_WORKERS_PER_KEY	1	Scripts marked at once per key. Keep 1 on free-tier keys.
SERVER_THREADS	8	Web server threads.

Setting	Default	Purpose
HOST / PORT	0.0.0.0 / 5000	Network address and port the app listens on.
UPLOAD_FOLDER	uploads/exams	Where sessions are stored.

24. Where data is stored

ExamManager uses no database. Everything is plain files in the project folder on the host, written safely (locked, atomic writes) so a crash can't leave a half-written file.

```

uploads/exams/<year>/<term>/<class>_<stream>/<subject>/<exam>/
├─ students_metadata.json      exam details + every student's result
├─ question_paper_1.pdf ...    question paper pages
├─ rubric_1.jpg ...            marking scheme pages
├─ rubric_text.txt             extra marking notes (if typed)
├─ student_<id>/
│   ├─ page_1.jpg ...          answer pages (as uploaded)
│   ├─ original.pdf            the uploaded PDF, if any
│   ├─ previous_uploads/       pages replaced by a re-upload
│   └─ ai_runs/*.json          every AI reply for this student

```

File or folder	Contents
users.json	Teacher accounts (hashed passwords).
pending_jobs.json	Marking jobs not yet finished (resumed on restart).
settings.json	Automatic clean-up setting.
deletion_log.json	Record of clean-up deletions (last 1,000).
logs/markings.log	Marking timings, tokens, retries.
backups/	Backup ZIPs.
benchmarks/	Benchmark results.
<exam>_backup_<time>	A session that was replaced at setup (kept, hidden from Results).

25. Security and privacy

- **Sign-in required** for every page and API. Sessions expire after 12 hours; cookies are HTTP-only. Repeated wrong passwords are rate-limited.
- **Passwords** are stored only as salted hashes and typed at hidden prompts, never on the command line.
- **Secrets** (`.env` , API keys, `users.json`) and all student data (`uploads/`) are excluded from version control. Never commit or share them.
- **Upload safety:** folder names and IDs are validated so they can't point outside the data folder; ZIPs are checked for unsafe paths, encryption, zip bombs and corrupted files before anything is written.
- **Data leaves the school only to the AI service.** Question papers, marking schemes and answer-script images are sent to Google's Gemini API for marking. Student names and IDs appear in the request. Make sure this is acceptable under your school's and country's data-protection rules, and consider the key's Google Cloud terms.
- **Network:** the app is meant for the school's local network and serves plain HTTP. Don't expose port 5000 directly to the internet. If remote access is needed, put it behind a VPN or an HTTPS reverse proxy.
- **Retention:** use automatic clean-up and backup rotation to control how long student scripts are kept.

26. Troubleshooting

Failure reasons shown on a student

Reason shown	Cause	Fix
Pages unreadable or blank	The AI found no answers on the pages.	Re-capture and upload the student again.
No readable pages found	The student's page files are missing.	Upload the student's pages again.
Question paper or marking scheme missing	Session materials not found on disk.	Re-run setup for the exam with the paper and scheme.
Daily AI limit reached on every key	All keys are out of quota.	Retry failed later, or add keys from other projects.
AI service busy / Could not reach the AI service	Overload or network problem that persisted through retries.	Check the host's internet, then Retry failed.
Report incomplete / AI reply could not be read	The AI's reply was cut off or in the wrong format.	Retry failed (re-mark).
AI service not set up	No API keys in <code>.env</code> .	Administrator adds keys and restarts.

Common questions

▼ Teachers can't open the app from their phones.

Make sure the phone is on the same Wi-Fi as the host, use the `192.168.x.x` address printed at start-up (not `localhost`), and allow Python through the host's firewall on port 5000. The host's IP address can change after a restart; check the console.

▼ The camera doesn't start.

Allow camera access when the browser asks. Some browsers only allow the camera on secure (HTTPS) pages or on `localhost`; if the camera is blocked on the school network

address, use **Upload files** and pick photos taken with the phone's camera app instead.

▼ **An upload is rejected as too blurry, but the page looks fine.**

The check is strict on purpose. Retake in brighter, even light with the phone held steady and parallel to the page. Very pale or very dark photos are also rejected. Scanned PDFs are not blur-checked.

▼ **"This exam session is no longer available."**

The exam's folder was moved, replaced or deleted. Set the exam up again, or pick it from Results and use Add students.

▼ **Marking seems stuck.**

Check the Overview queue panel: it may be paused (resume it), retrying because the AI is busy, or waiting because every key is resting. Check the host is awake and online, and look at `logs/marking.log`.

▼ **The total marks were wrong.**

Straight after setup, before anyone is marked, choose *Go back and edit* in the total-marks check. Once students are marked with the old total it can't be changed there; set up the exam again with *Replace it*, or correct individual marks with Edit.

▼ **How do I re-mark a student?**

Upload their pages again with the same ID and name. Their previous mark is kept in the history. (Students with a teacher's mark are not re-marked.)

▼ **Sign-in says "Too many attempts".**

Wait five minutes and try again. If the password is forgotten, the administrator resets it with `python manage_users.py passwd <username>`.

27. Glossary

Session / exam	One paper for one class and stream: year + term + class + stream + subject + exam name.
Script	A student's answer pages.
Marking scheme / rubric	The official guide listing what earns marks for each question.
Stage	One independently scored part of the scheme, e.g. Q1(a).
Raw marks	Sum of stage marks, out of the paper's total marks.
Score / percentage	Raw marks ÷ total marks × 100.
Override	A teacher-set final mark that replaces the AI's mark; the AI's mark is kept on record.
Queue / job	The list of scripts waiting to be marked; each script is one job.
API key	A credential for Google's Gemini AI service. Each key adds a marking worker.
Batch	A whole class uploaded at once as a ZIP or folder.
Benchmark	A consistency test used before changing AI settings.

ExamManager project guide. Covers the application as found in the project repository: web app (`app.py`), AI marker, marking queue, batch processor, account, backup and benchmark tools.